Singleton

Singleton · One MetricsRegistry across request threads · Build with Waqar

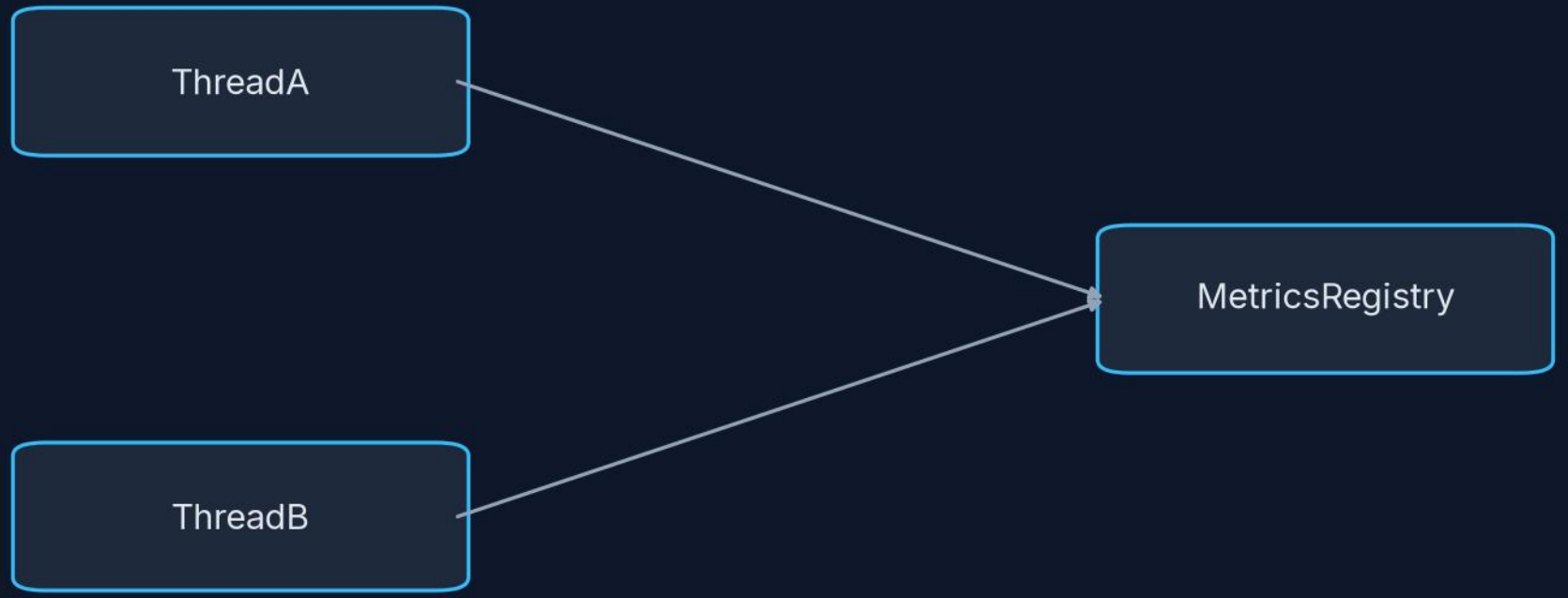
Today's Hook

- Problem: One MetricsRegistry across request threads
- Pattern: Singleton — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- One MetricsRegistry across request threads
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Singleton: One MetricsRegistry across request threads

Real-World Scenario

- Running example: One MetricsRegistry across request threads
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.singleton;

import com.buildwithwaqar.patterns.gof.creational.singleton.problem.NaiveMetrics;

/** Problem: one MetricsRegistry across request threads (vs split-brain counters). */
public final class UseCaseDemo {
    public static void main(String[] args) {
        System.out.println("=== Singleton problem: one metrics registry ===");
        var a = new NaiveMetrics();
        var b = new NaiveMetrics();
        a.hit(); a.hit(); b.hit();
        System.out.println("naiveA=" + a.hits() + " naiveB=" + b.hits() + " (split brain)");

        MetricsRegistry.getInstance().reset();
        MetricsRegistry.getInstance().hit();
        MetricsRegistry.getInstance().hit();
        MetricsRegistry.getInstance().hit();
        System.out.println("singletonHits=" + MetricsRegistry.getInstance().hits());
        System.out.println("same=" + (MetricsRegistry.getInstance() == MetricsRegistry.getInstance()));
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.creational.singleton;

/** Preferred: enum singleton — serialization-safe, reflection-safe. */
public enum AppConfig {
    INSTANCE;

    private String env = "prod";

    public String env() { return env; }
    public void setEnv(String env) { this.env = env; }
    public String greeting() { return "hello from " + env; }
}
```

```java
package com.buildwithwaqar.patterns.gof.creational.singleton;

import java.time.Instant;

/** Prefer injecting a Clock over a time singleton. */
public interface Clock {
    Instant now();
```

Solution (2)

```
    record SystemClock() implements Clock {
        @Override public Instant now() { return Instant.now(); }
    }

    record FixedClock(Instant instant) implements Clock {
        @Override public Instant now() { return instant; }
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.creational.singleton;

import com.buildwithwaqar.patterns.gof.creational.singleton.problem.NaiveMetrics;

/** Problem: one MetricsRegistry across request threads (vs split-brain counters). */
public final class UseCaseDemo {
    public static void main(String[] args) {
        System.out.println("=== Singleton problem: one metrics registry ===");
        var a = new NaiveMetrics();
        var b = new NaiveMetrics();
        a.hit(); a.hit(); b.hit();
        System.out.println("naiveA=" + a.hits() + " naiveB=" + b.hits() + " (split brain)");

        MetricsRegistry.getInstance().reset();
        MetricsRegistry.getInstance().hit();
        MetricsRegistry.getInstance().hit();
        MetricsRegistry.getInstance().hit();
        System.out.println("singletonHits=" + MetricsRegistry.getInstance().hits());
        System.out.println("same=" + (MetricsRegistry.getInstance() == MetricsRegistry.getInstance()));
    }
}
```

Walkthrough Output

```
=== Singleton problem: one metrics registry ===
naiveA=2 naiveB=1 (split brain)
singletonHits=3
same=true
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Singleton.

Related Patterns

- Contrast Singleton with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Singleton → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.